SI252 Reinforcement Learning - Homework 2

Complete solutions: derivations, algorithms, reproducible experiments, and analysis
Assignment date: October 8, 2026. Due: October 18, 2026, 10:59 pm (as stated in the assignment).

All questions are answered in order. For Questions 1-3, the samplers are implemented directly; SciPy supplies theoretical PDF/PMF, CDF, and statistical diagnostics, rather than random samples from the target distributions. Question 4 includes analytical proofs and exact fractions, followed by optional numerical verification.

Running this notebook: Use Python 3 with `numpy scipy matplotlib nbformat ipykernel`, and run all cells from top to bottom. A fixed seed makes the simulations reproducible. Each main sampling method generates 100,000 samples. Timing measurements depend on the machine.

Envelope terminology: For rejection sampling, the envelope is $Mq(x)$. Inverse-CDF and Box-Muller sampling do not use a rejection envelope, so their plots explicitly mark it as N/A. A theoretical density curve is not itself called a rejection envelope.

In [1]:
```
import sys, platform
from time import perf_counter
from fractions import Fraction as F
import numpy as np
import scipy
from scipy import stats
import matplotlib
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

SEED = 2522026
N = 100_000
rng = np.random.default_rng(SEED)
plt.rcParams.update({'figure.dpi': 120, 'axes.grid': True, 'grid.alpha': .2,
                     'axes.spines.top': False, 'axes.spines.right': False})
print('Python:', platform.python_version())
print('NumPy:', np.__version__, '| SciPy:', scipy.__version__, '| Matplotlib:', mat
print('Seed:', SEED, '| Samples per method:', N)

def open_uniform(size, generator=rng):
    # Exclude 0 and 1 before logarithms; clipping affects only floating-point endpo
    return np.clip(generator.random(size), np.nextafter(0., 1.), np.nextafter(1., 0

def table(headers, rows):
```

```
    headers = [str(v).replace('|', '&#124;') for v in headers]
    rows = [[str(v).replace('|', '&#124;') for v in row] for row in rows]
    display(Markdown('| ' + ' | '.join(map(str, headers)) + ' |\n| ' +
                     ' | '.join(['---'] * len(headers)) + ' |\n' +
                     '\n'.join('| ' + ' | '.join(map(str, row)) + ' |' for row in r

def continuous_plot(ax, samples, dist, title, limits, envelope=None):
    # Density is normalized using ALL samples; limits only crop the displayed view.
    ax.hist(samples, bins=160, density=True, alpha=.45, color='#4996c4', label='Emp
    grid = np.linspace(*limits, 1000)
    ax.plot(grid, dist.pdf(grid), color='#bd3434', lw=2, label='Theoretical PDF')
    if envelope is not None:
        ax.plot(grid, envelope(grid), '--', color='#333333', lw=1.7, label='Envelop
    ax.set(xlim=limits, title=title, xlabel='x', ylabel='Density')
    if envelope is None:
        ax.text(.98, .96, 'Rejection envelope: N/A', transform=ax.transAxes,
                ha='right', va='top', fontsize=8)
    ax.legend(fontsize=8)

def continuous_diagnostics(items):
    rows=[]
    for name, x, dist in items:
        ks = stats.kstest(x, dist.cdf)
        rows.append([name, f'{np.mean(x):.5f}', f'{float(dist.mean()):.5f}',
                     f'{np.var(x, ddof=1):.5f}', f'{float(dist.var()):.5f}',
                     f'{ks.statistic:.5f}', f'{ks.pvalue:.4f}'])
    table(['Method','Empirical mean','True mean','Empirical variance','True varianc
```

```
Python: 3.12.7
NumPy: 1.26.4 | SciPy: 1.13.1 | Matplotlib: 3.9.2
Seed: 2522026 | Samples per method: 100000
```

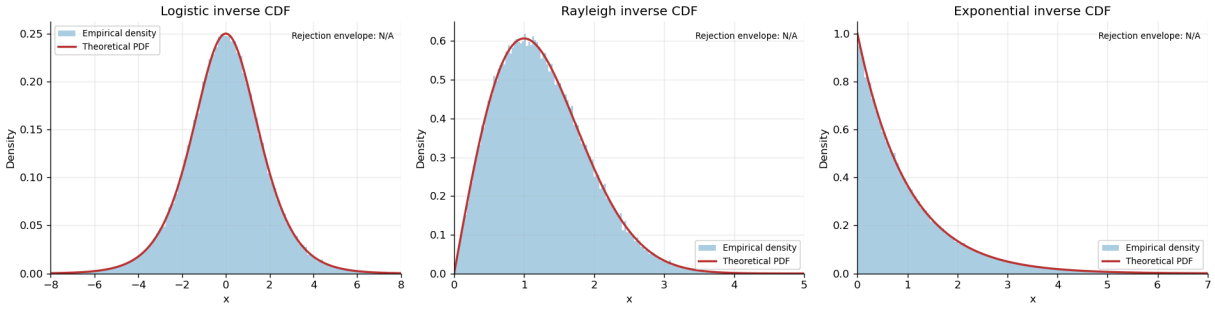
1. Continuous distributions: inverse transform sampling

Let $U\sim\mathrm{Unif}(0,1)$. For a continuous, strictly increasing CDF, set $X=F^{-1}(U)$. Then $$P(X\le x)=P(U\le F(x))=F(x).$$ Solving $u=F(x)$ gives the following transformations.

Part	Sampling transformation	Theoretical PDF	Mean; variance
(a) Logistic	$X=\log U-\log(1-U)$	$e^{-x}/(1+e^{-x})^2,\ x\in\mathbb R$	$0;\ \pi^2/3$
(b) Rayleigh	$X=\sqrt{-2\log(1-U)}$	$xe^{-x^2/2},\ x>0$	$\sqrt{\pi/2};\ (4-\pi)/2$
(c) Exponential	$X=-\log(1-U)$	$e^{-x},\ x>0$	$1;\ 1$

Algorithm: Independently draw one uniform variate for each sample and apply the relevant transformation. `log1p(-u)` computes $\log(1-u)$ stably. Generating n samples takes $O(n)$ time and requires no rejection.

In [2]:
```
u = open_uniform(N)
x_logistic = np.log(u) - np.log1p(-u)
x_rayleigh = np.sqrt(-2 * np.log1p(-open_uniform(N)))
x_exp = -np.log1p(-open_uniform(N))
continuous_items = [('Logistic inverse CDF', x_logistic, stats.logistic()),
                    ('Rayleigh inverse CDF', x_rayleigh, stats.rayleigh()),
                    ('Exponential inverse CDF', x_exp, stats.expon())]
fig, axes = plt.subplots(1, 3, figsize=(16, 4.2))
for ax, (name, x, dist), lim in zip(axes, continuous_items, [(-8, 8), (0, 5), (0, 7
    continuous_plot(ax, x, dist, name, lim)
plt.tight_layout(); plt.show()
continuous_diagnostics(continuous_items)
```



Method	Empirical mean	True mean	Empirical variance	True variance	KS D	KS p
Logistic inverse CDF	0.00294	0.00000	3.27304	3.28987	0.00151	0.9758
Rayleigh inverse CDF	1.25376	1.25331	0.42673	0.42920	0.00197	0.8313
Exponential inverse CDF	1.00313	1.00000	1.00138	1.00000	0.00327	0.2343

Results and analysis. The sample means for Logistic, Rayleigh, and Exponential are 0.00294, 1.25376, and 1.00313; their KS distances are 0.00151, 0.00197, and 0.00327. The histograms display the symmetric Logistic tails, the positive unimodal Rayleigh shape, and the decreasing exponential density. All samples enter the moment estimates and histogram normalization; the displayed axis limits do not truncate the sampling distribution.

The Kolmogorov-Smirnov test compares the entire empirical CDF with the theoretical CDF and is appropriate here because the continuous-distribution parameters are fixed in advance. Random samples can still produce a small p-value. Failure to reject is not a proof of correctness; the inverse-transform argument provides that proof.

2. Discrete distributions: a general inverse-CDF algorithm

Let the ordered support be $x_0<x_1<\cdots$, let $p_k=P(X=x_k)$, and define $C_k=\sum_{j=0}^kp_j$, with $C_{-1}=0$. Draw $U\sim\mathrm{Unif}(0,1)$ and set $$K=\min\{k:U\le C_k\},\qquad X=x_K.$$ Since $P(C_{k-1}<U\le C_k)=C_k-C_{k-1}=p_k$, this algorithm is correct. Zero-mass entries cause no problem; a uniform draw exactly on a boundary has probability zero.

For finite support of size m, cumulative probabilities take $O(m)$ preprocessing time, followed by $O(\log m)$ binary search per sample. For infinite support, accumulate until the CDF exceeds the current uniform draw. The batch implementation below extends the genuine CDF beyond the largest uniform draw in the batch: it neither truncates nor renormalizes the distribution.

(a)-(d). PMFs, recurrences, and counting conventions

- **Bernoulli(0.5):** $P(X=0)=P(X=1)=1/2$, so $X=\mathbf1\{U>1/2\}$. The mean is $1/2$ and the variance is $1/4$.
- **Binomial(20,0.5):** $P(X=k)=\binom{20}{k}2^{-20}$ for $0\le k\le20$. Start with $p_0=2^{-20}$ and use $p_{k+1}/p_k=(20-k)/(k+1)$. The mean is 10 and the variance is 5.
- **Geom(0.5):** Here X counts trials up to and including the first success: $P(X=k)=2^{-k}$ for $k\ge1$. Since $F(k)=1-2^{-k}$, use $X=\lceil\log(1-U)/\log(1/2)\rceil$. The mean and variance are both 2. Under the alternative failures-before-success convention, use $X-1$, whose mean is 1.
- **NBin(10,0.5):** Here X counts failures before the tenth success: $P(X=k)=\binom{k+9}{9}2^{-(k+10)}$ for $k\ge0$. Start with $p_0=2^{-10}$ and use $p_{k+1}/p_k=(k+10)/(2(k+1))$. The mean is 10 and the variance is 20. If the variable instead counts total trials to the tenth success, use $X+10$; its support starts at 10 and its variance is unchanged.

These conventions are stated explicitly because geometric and negative-binomial distribution names alone do not uniquely specify the counted quantity.

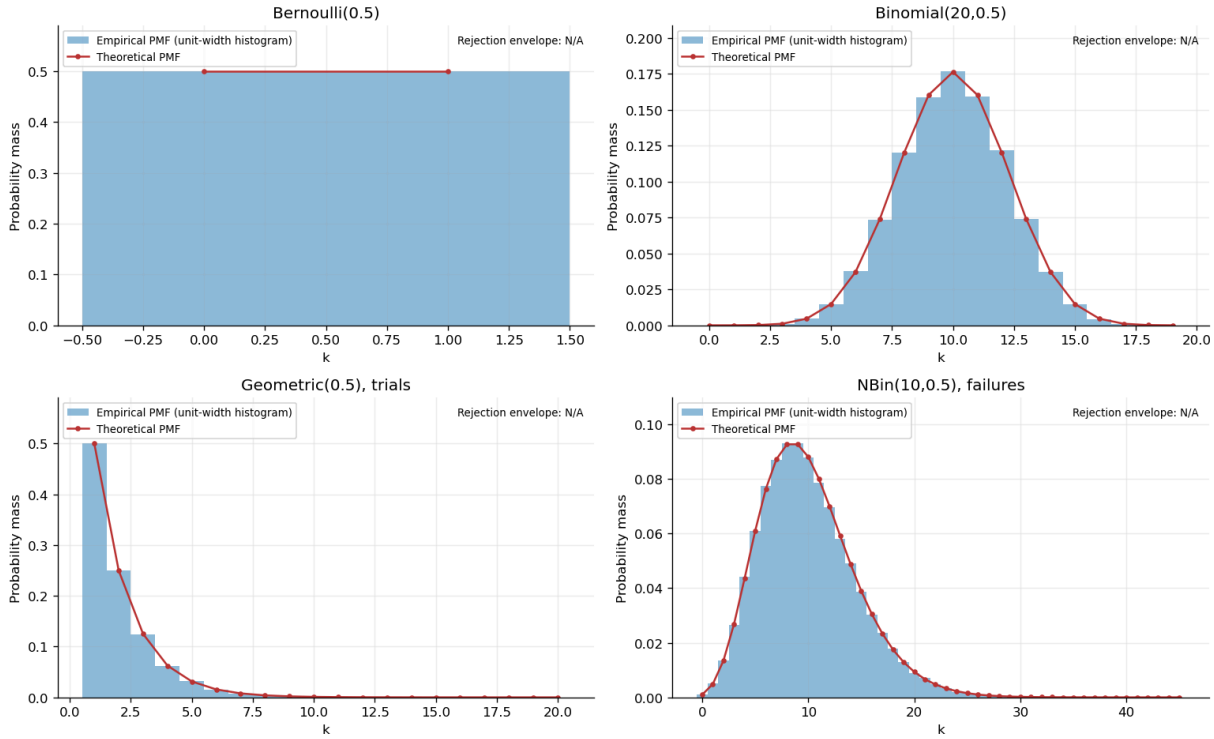
In [3]:
```python
def inverse_finite(support, probabilities, u):
    support = np.asarray(support)
    probabilities = np.asarray(probabilities, dtype=float)
    assert len(support) == len(probabilities)
    assert np.all(probabilities >= 0) and np.isclose(probabilities.sum(), 1)
    cdf = np.cumsum(probabilities)
    cdf[-1] = 1.0  # remove floating-point summation drift on finite support
    return support[np.searchsorted(cdf, u, side='left')]

def inverse_nbin_failures(u, r=10, p=.5):
    # Grow a genuine CDF until every uniform draw is covered.
    masses = [p**r]
    cumulative = [masses[0]]
    k = 0
    while cumulative[-1] < np.max(u):
```

```
        next_mass = masses[-1] * (k+r)/(k+1) * (1-p)
        next_cdf = cumulative[-1] + next_mass
        if next_cdf == cumulative[-1]:
            raise FloatingPointError('CDF accumulation stalled; use higher precisio
        masses.append(next_mass)
        cumulative.append(next_cdf)
        k += 1
    return np.searchsorted(cumulative, u, side='left'), cumulative[-1], k

x_bern = inverse_finite([0, 1], [.5, .5], open_uniform(N))
bin_pmf = np.empty(21); bin_pmf[0] = 2.**-20
for k in range(20):
    bin_pmf[k+1] = bin_pmf[k]*(20-k)/(k+1)
x_bin = inverse_finite(np.arange(21), bin_pmf, open_uniform(N))
x_geom = np.ceil(np.log1p(-open_uniform(N))/np.log(.5)).astype(int)
x_nb, nb_covered, nb_max = inverse_nbin_failures(open_uniform(N))
print(f'Adaptive negative-binomial CDF ends at k={nb_max}, F(k)={nb_covered:.12f};
discrete_items = [('Bernoulli(0.5)', x_bern, stats.bernoulli(.5)),
                  ('Binomial(20,0.5)', x_bin, stats.binom(20,.5)),
                  ('Geometric(0.5), trials', x_geom, stats.geom(.5)),
                  ('NBin(10,0.5), failures', x_nb, stats.nbinom(10,.5))]
fig, axes = plt.subplots(2, 2, figsize=(13, 8))
rows=[]
for ax, (name, x, dist) in zip(axes.flat, discrete_items):
    low, high = int(dist.support()[0]), int(max(x.max(), dist.ppf(.9999)))
    support = np.arange(low, high+1)
    empirical = np.bincount(x-low, minlength=len(support))/len(x)
    ax.hist(x, bins=np.arange(low-.5, high+1.5), weights=np.ones(len(x))/len(x),
            alpha=.5, label='Empirical PMF (unit-width histogram)')
    ax.plot(support, dist.pmf(support), 'o-', ms=3, color='#bd3434', label='Theoret
    ax.set(title=name, xlabel='k', ylabel='Probability mass')
    ax.set_ylim(0, 1.18*max(empirical.max(), dist.pmf(support).max()))
    ax.text(.98,.94,'Rejection envelope: N/A',transform=ax.transAxes,ha='right',fon
    ax.legend(fontsize=8)
    # Include all unobserved theoretical upper-tail mass in empirical TV.
    tv = .5*(np.abs(empirical-dist.pmf(support)).sum()+dist.sf(high))
    rows.append([name, f'{x.mean():.5f}', f'{dist.mean():.5f}',
                 f'{x.var(ddof=1):.5f}', f'{dist.var():.5f}', f'{tv:.5f}'])
plt.tight_layout(); plt.show()
table(['Distribution','Sample mean','True mean','Sample variance','True variance','
assert set(np.unique(x_bern)) <= {0,1}
assert np.all((0 <= x_bin) & (x_bin <= 20))
assert np.all(x_geom >= 1) and np.all(x_nb >= 0)
```

```
Adaptive negative-binomial CDF ends at k=45, F(k)=0.999999783181; no renormalizatio
n.
```



Distribution	Sample mean	True mean	Sample variance	True variance	Empirical TV
Bernoulli(0.5)	0.49993	0.50000	0.25000	0.25000	0.00007
Binomial(20,0.5)	9.99979	10.00000	4.99552	5.00000	0.00358
Geometric(0.5), trials	1.99707	2.00000	1.98908	2.00000	0.00192
NBin(10,0.5), failures	9.99668	10.00000	20.13735	20.00000	0.00485

Results and analysis. The empirical TV distances are 0.00007, 0.00358, 0.00192, and 0.00485, respectively, and the empirical means are close to the theoretical means. Histogram heights are sample frequencies; unit-width bins make them directly comparable to PMFs. Bernoulli has two nearly equal bars, Binomial concentrates around 10, Geometric decreases rapidly from 1, and Negative Binomial is right-skewed and more dispersed than Binomial. The empirical TV calculation includes the unobserved theoretical upper tail. Continuous-distribution KS p-values are not used for discrete data.

3. Acceptance-rejection and Box-Muller sampling

General correctness of rejection sampling

Suppose the target density satisfies $f\le Mq$. Independently draw $Y\sim q$ and $U\sim\mathrm{Unif}(0,1)$, accepting if $U\le f(Y)/(Mq(Y))$. For any measurable set B, $$P(Y\in B,\mathrm{accept})=\int_Bq(y)\frac{f(y)}{Mq(y)}dy=\frac1M\int_B

f(y)dy.$$ Integrating over the support gives $P(\mathrm{accept})=1/M$; conditioning on acceptance gives the target density f.

3(a). Beta(2,4)

$$f(x)=\frac{\Gamma(6)}{\Gamma(2)\Gamma(4)}x(1-x)^3=20x(1-x)^3,\quad0<x<1.$$ Choose the uniform proposal $q(x)=1$ on $(0,1)$. Since $f'(x)=20(1-x)^2(1-4x)$, the global maximum occurs at $x=1/4$: $$M=f(1/4)=\frac{135}{64},\qquad P(\mathrm{accept})=\frac{64}{135}.$$ Draw $Y\sim\mathrm{Unif}(0,1)$ and accept with probability $20Y(1-Y)^3/M$. The expected number of proposals per output is $135/64$. The target mean is $1/3$ and variance is $2/63$.

3(b). Two standard Normal samplers

Box-Muller: Independently draw $U_1,U_2\sim\mathrm{Unif}(0,1)$ and define $$R=\sqrt{-2\log U_1},\quad\Theta=2\pi U_2,\quad Z_1=R\cos\Theta,\quad Z_2=R\sin\Theta.$$ Two uniform variates produce two independent standard Normal variates, with no rejection.

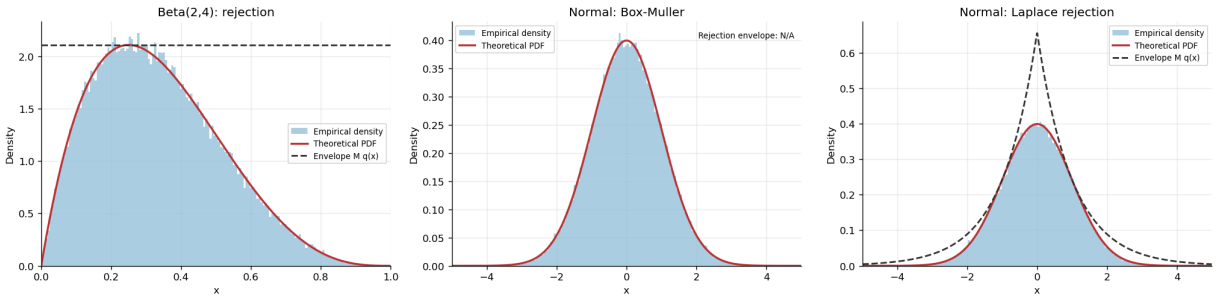
Acceptance-rejection: Choose the Laplace proposal $q(x)=\tfrac12e^{-|x|}$. Generate it by inverse transform: if $u<1/2$, let $Y=\log(2u)$; otherwise let $Y=-\log(2(1-u))$. Then $$\frac{\phi(x)}{q(x)}=\sqrt{\frac2\pi}\exp(-x^2/2+|x|)=\sqrt{\frac{2e}\pi}\exp[-(|x|-1)^2/2].$$ The smallest valid envelope constant is $M=\sqrt{2e/\pi}$. Thus accept if $$U\le\exp[-(|Y|-1)^2/2],\qquad P(\mathrm{accept})=\sqrt{\frac\pi{2e}}.$$ The Laplace tails are heavier than Normal tails, so this envelope is valid on the entire real line. A uniform proposal on a bounded interval cannot envelope the full Normal density.

In [4]:
```
def rejection_sample(n, propose, accept_probability, generator=rng):
    result = []
    kept = proposals = accepted_total = 0
    while kept < n:
        batch_size = max(1024, n-kept)
        y = propose(batch_size, generator)
        accepted = y[generator.random(batch_size) <= accept_probability(y)]
        result.append(accepted[:n-kept])
        kept += min(len(accepted), n-kept)
        proposals += batch_size
        accepted_total += len(accepted)
    # Include surplus accepted proposals in the acceptance-rate denominator/numerat
    return np.concatenate(result), proposals, accepted_total

M_beta = 135/64
M_normal = np.sqrt(2*np.e/np.pi)
def propose_uniform(n, generator):
    return open_uniform(n, generator)
def propose_laplace(n, generator):
    u = open_uniform(n, generator)
```

```
    return np.where(u < .5, np.log(2*u), -np.log(2*(1-u)))
def beta_accept(y):
    return 20*y*(1-y)**3/M_beta
def normal_accept(y):
    return np.exp(-.5*(np.abs(y)-1)**2)
def box_muller(n, generator=rng):
    pairs = (n+1)//2
    radius = np.sqrt(-2*np.log(open_uniform(pairs,generator)))
    theta = 2*np.pi*open_uniform(pairs,generator)
    return np.column_stack((radius*np.cos(theta), radius*np.sin(theta))).ravel()[:n

x_beta, beta_proposals, beta_accepted = rejection_sample(N, propose_uniform, beta_a
x_bm = box_muller(N)
x_normal_ar, normal_proposals, normal_accepted = rejection_sample(N, propose_laplac
fig, axes = plt.subplots(1, 3, figsize=(17,4.3))
continuous_plot(axes[0], x_beta, stats.beta(2,4), 'Beta(2,4): rejection', (0,1),
                lambda x: np.full_like(x,M_beta))
continuous_plot(axes[1], x_bm, stats.norm(), 'Normal: Box-Muller', (-5,5))
continuous_plot(axes[2], x_normal_ar, stats.norm(), 'Normal: Laplace rejection', (-
                lambda x: M_normal*.5*np.exp(-np.abs(x)))
plt.tight_layout(); plt.show()
continuous_diagnostics([('Beta rejection',x_beta,stats.beta(2,4)),
                        ('Normal Box-Muller',x_bm,stats.norm()),
                        ('Normal rejection',x_normal_ar,stats.norm())])
rows=[]
for name, proposals, accepted, M in [('Beta',beta_proposals,beta_accepted,M_beta),
                                     ('Normal',normal_proposals,normal_accepted,M_n
    rate=accepted/proposals
    se=np.sqrt((1/M)*(1-1/M)/proposals)
    rows.append([name, proposals, accepted, f'{rate:.6f}', f'{1/M:.6f}',
                 f'{(rate-1/M)/se:.2f}', f'{M:.6f}'])
table(['Rejection sampler','All proposals','All accepted','Observed rate','Theory r
# Deterministic envelope checks supplement the analytic global proof.
grid=np.linspace(0,1,10001)
assert np.all(stats.beta(2,4).pdf(grid) <= M_beta+1e-12)
grid=np.linspace(-15,15,30001)
assert np.all(stats.norm.pdf(grid) <= M_normal*.5*np.exp(-np.abs(grid))+1e-12)
pair=x_bm.reshape(-1,2)
print('Box-Muller within-pair correlation:', np.corrcoef(pair.T)[0,1])
```



Method	Empirical mean	True mean	Empirical variance	True variance	KS D	KS p
Beta rejection	0.33335	0.33333	0.03160	0.03175	0.00293	0.3569
Normal Box-Muller	0.00187	0.00000	1.00165	1.00000	0.00167	0.9427
Normal rejection	0.00127	0.00000	1.01028	1.00000	0.00268	0.4689

Rejection sampler	All proposals	All accepted	Observed rate	Theory rate	Deviation / SE	Theory proposals/output
Beta	211479	100397	0.474737	0.474074	0.61	2.109375
Normal	132410	100427	0.758455	0.760173	-1.46	1.315489

```
Box-Muller within-pair correlation: -0.0019211564522420817
```

3(c). Proof of the Box-Muller method

For $r>0$, $$P(R\le r)=P(U_1\ge e^{-r^2/2})=1-e^{-r^2/2}.$$ Consequently, $$f_R(r)=re^{-r^2/2},\quad r>0;\qquad f_\Theta(\theta)=1/(2\pi),\quad0\le\theta<2\pi.$$ The variables R and Θ are independent because U_1 and U_2 are independent. The polar transformation $(z_1,z_2)=(r\cos\theta,r\sin\theta)$ has absolute Jacobian determinant r. Apart from the origin and angular boundary, which have probability zero, $$f_{Z_1,Z_2}(z_1,z_2)=\frac{r e^{-r^2/2}}{2\pi r} =\frac{e^{-z_1^2/2}}{\sqrt{2\pi}}\frac{e^{-z_2^2/2}}{\sqrt{2\pi}}.$$ The joint density factors into two standard Normal densities, proving both Normality and independence. The reported sample correlation is an auxiliary diagnostic, not a proof of independence.

Comparison and experimental analysis

Method	Advantages	Costs and limitations
Box-Muller	No rejection; two uniforms give two Normals; easily vectorized	Uses log, sqrt, sin, and cos; cache the second output when requesting samples individually
Laplace rejection	Simple acceptance expression; illustrates a general method; theoretical acceptance rate about 0.760	About 1.315 proposals per output on average; additional uniforms and a rejection loop
Uniform-proposal Beta rejection	Simple proposal and envelope	Acceptance rate about 0.474; uniform envelopes become inefficient for highly concentrated targets

The observed Beta acceptance rate is 0.474737 and the Normal rejection rate is 0.758455, approximately 0.61 and -1.46 standard errors from theory. The two Normal

samplers have sample means 0.00187 and 0.00127, with sample variances 1.00165 and 1.01028. Density plots, moment estimates, and acceptance rates jointly check the implementation.

Theoretical proposal efficiency does not directly determine execution time. The following repeated benchmark includes random-number generation and array operations, but excludes plotting. Its results describe this implementation and environment rather than a universal performance ranking.

In [5]:
```
timings = {}
for name in ['Box-Muller','Laplace rejection']:
    durations=[]
    for repeat in range(7):
        local_rng=np.random.default_rng(SEED+100+repeat)
        start=perf_counter()
        if name == 'Box-Muller':
            _=box_muller(N,local_rng)
        else:
            _=rejection_sample(N,propose_laplace,normal_accept,local_rng)
        durations.append(perf_counter()-start)
    timings[name]=float(np.median(durations))
table(['Method','Median of 7 runs (ms)','Samples / second'],
      [[name,f'{t*1000:.3f}',f'{N/t:,.0f}'] for name,t in timings.items()])
```

Method	Median of 7 runs (ms)	Samples / second
Box-Muller	2.334	42,844,901
Laplace rejection	5.358	18,662,287

4. From acceptance-rejection to lossless speculative decoding

The vocabulary is $\mathcal V=\{A,B,C,D\}$, with distributions:

Token	Target $p(x)$	Draft $q(x)$
A	$2/5$	$1/5$
B	$3/10$	$2/5$
C	$1/5$	$1/10$
D	$1/10$	$3/10$

4(a). Uniformity under restriction

1. For any measurable $C\subseteq S$, $$P(Z\in C\mid Z\in S)=\frac{P(Z\in C)}{P(Z\in S)} =\frac{|C|/|R|}{|S|/|R|}=\frac{|C|}{|S|}.$$ This is precisely the defining property of

$\mathrm{Unif}(S)$. For an arbitrary measurable set B, apply the same argument to $C=B\cap S$.

2. Every unit of area initially has the same probability density. Removing points outside S changes only the common normalization constant, preserving relative probabilities within S. For sampling a nonuniform density, draw uniformly under $Mq(x)$ and keep points lying under $p(x)$. The retained points are uniform under the target curve; projecting onto the horizontal axis yields probabilities proportional to the target's vertical strip areas.

4(b). Classical acceptance-rejection

1-2. The target-to-proposal ratios and minimum envelope constant are $$\left(\frac{p(A)}{q(A)},\frac{p(B)}{q(B)},\frac{p(C)}{q(C)},\frac{p(D)}{q(D)}\right) = (2,3/4,2,1/3),\qquad M^*=2.$$

Token	$a_{\rm classical}(x)=p(x)/(2q(x))$	$q(x)a_{\rm classical}(x)$
A	1	$1/5$
B	$3/8$	$3/20$
C	1	$1/10$
D	$1/6$	$1/20$

3. Summing the accepted masses gives $$P(\mathrm{accept})=\sum_xq(x)\frac{p(x)}{M^*q(x)}=\frac1{M^*}\sum_xp(x)=\frac1{M^*}=\frac12.$$ Conditioned on acceptance, the output probability is $(p(x)/2)/(1/2)=p(x)$.

4. Independent repetitions have success probability $s=1/2$. The number of proposals $T\in\{1,2,\ldots\}$ has first-success distribution $P(T=t)=(1-s)^{t-1}s$. Therefore, $$E[T]=\sum_{t\ge1}P(T\ge t)=\sum_{t\ge1}(1-s)^{t-1}=1/s=2.$$

4(c). Overlap acceptance

1-2. Define $a(x)=\min(1,p(x)/q(x))$ on the proposal support. If $p(x)\ge q(x)>0$, the ratio is at least one, so the token is always accepted. Moreover, $$P(Y=x,\mathrm{accept})=q(x)\min(1,p(x)/q(x))=\min(p(x),q(x)).$$

Token	$a(x)$	Accepted mass $m(x)$
A	1	$1/5$
B	$3/4$	$3/10$
C	1	$1/10$
D	$1/3$	$1/10$

Thus $\alpha=1/5+3/10+1/10+1/10=7/10$.

3. $P(Y=A\mid\mathrm{accept})=(1/5)/(7/10)=2/7\ne2/5=p(A)$. Accepted tokens have distribution $m(x)/\alpha$, which generally differs from $p(x)$. Repeatedly proposing until this overlap rule accepts would therefore not produce exact target samples.

4. Represent each token by a unit-width strip with heights $p(x)$ and $q(x)$. The intersection height, and hence its area, is $m(x)=\min(p(x),q(x))$. A uniform point in R_q can be generated by drawing $Y\sim q$, a uniform horizontal location within its strip, and height $H\mid Y\sim\mathrm{Unif}(0,q(Y))$. Accepting when $H\le p(Y)$ restricts the point to $R_p\cap R_q$. By (a), retained points are uniform on this intersection, and their token probabilities are intersection strip areas divided by total intersection area: $m(x)/\alpha$.

4(d). Residual correction

1. Set $d(x)=(p(x)-q(x))_+$. Direct calculation gives $$d=(1/5,0,1/10,0),\quad\delta=\sum_xd(x)=3/10,\quad r=(2/3,0,1/3,0).$$

2. Since $q(x)-\min(p(x),q(x))=(q(x)-p(x))_+$, $$P(\mathrm{reject})=1-\alpha=\sum_x(q(x)-p(x))_+.$$ Also $\sum_x(p(x)-q(x))=0$, so total positive and negative deviations are equal. Hence $$\sum_x(q(x)-p(x))_+=\sum_x(p(x)-q(x))_+=\delta=3/10.$$

3. For arbitrary probability distributions p,q on a finite vocabulary, the identity $$p(x)=\min(p(x),q(x))+(p(x)-q(x))_+$$ holds by considering $p(x)\le q(x)$ and $p(x)>q(x)$ separately. When $\delta>0$, draw a replacement independently from $r=d/\delta$ after rejection. Then $$P(X=x)=P(Y=x,\mathrm{accept})+P(\mathrm{reject})r(x)=\min(p(x),q(x))+\delta\frac{(p(x)-q(x))_+}{\delta}=p(x).$$ If $q(x)=0$, the draft never proposes x, so its acceptance ratio need not be evaluated; the residual supplies any missing target mass. If $\delta=0$, normalization implies $p=q$, no rejection occurs, and no residual distribution is needed. Thus the proof covers zero-probability cases as well.

4.

Token	Accepted mass	Residual contribution $\delta r(x)$	Final output probability
A	$1/5$	$1/5$	$2/5$
B	$3/10$	0	$3/10$
C	$1/10$	$1/10$	$1/5$
D	$1/10$	0	$1/10$

The final column equals $p(x)$ token by token. Draft errors affect acceptance efficiency but not the corrected output distribution. This is the lossless property under exact probability arithmetic.

4(e). Total variation and maximal coupling

1. Using $\min(p,q)=(p+q-|p-q|)/2$ and summing, $$\alpha=\sum_x\min(p(x),q(x))=1-\frac12\sum_x|p(x)-q(x)|=1-D_{\rm TV}(p,q).$$ Here $D_{\rm TV}=\tfrac12(1/5+1/10+1/10+1/5)=3/10$, so $\alpha=7/10$, agreeing with (c).

2. For any coupling (X,Y) with marginals p,q, $$P(X=x,Y=x)\le P(X=x)=p(x),\qquad P(X=x,Y=x)\le P(Y=x)=q(x).$$ The diagonal events are disjoint, so $P(X=Y)\le\sum_x\min(p(x),q(x))$.

3. In speculative sampling, acceptance sets $X=Y$. Rejection can occur only at tokens with $q(Y)>p(Y)$, whereas residual replacements have support only where $p(X)>q(X)$. These supports are disjoint, so rejection cannot yield $X=Y$. Therefore, $$P(X=Y)=P(\mathrm{accept})=\sum_x\min(p(x),q(x)).$$ Together with $X\sim p$ from (d) and $Y\sim q$ by construction, this attains the upper bound for all couplings and is therefore a maximal coupling. A draft closer to the target in TV distance has a greater probability of having its proposal reused. Missing draft support does not invalidate the construction.

4(f). Autoregressive sequences

Conditioned on the prefix $X_1=A$, the given distributions yield:

Token	$p(x\mid A)$	$q(x\mid A)$	Acceptance $a(x\mid A)$	Accepted mass	$d(x\mid A)$	$r(x\mid A)$
A	$1/2$	$1/4$	1	$1/4$	$1/4$	$1/2$
B	$1/4$	$1/2$	$1/2$	$1/4$	0	0
C	$1/4$	0	N/A (never proposed)	0	$1/4$	$1/2$
D	0	$1/4$	0	0	0	0

1. The draft can propose A,B,D, whose acceptance probabilities are $1,1/2,0$. Total acceptance is $1/4+1/4=1/2$, and rejection is $1/2$. The acceptance ratio for C is undefined and unnecessary because it cannot be proposed.

2. The unnormalized residual is $(1/4,0,1/4,0)$, giving $r=(1/2,0,1/2,0)$. Thus $$P(X_2=C\mid X_1=A)=0+\frac12\cdot\frac12=\frac14.$$ Although the draft never proposes C, residual correction supplies its exact target probability. The output probabilities of A,B,D are likewise $1/2,1/4,0$.

3. Using the original first-token probability $p(A)=2/5$, $$P(X_1=A,X_2=C)=P(X_1=A)P(X_2=C\mid X_1=A)=\frac25\cdot\frac14=\boxed{\frac1{10}}.$$ This agrees with direct autoregressive sampling from the target.

4. At every positive-probability history $h=x_{1:t-1}$, apply (d) to the conditional distributions $p(\cdot\mid h),q(\cdot\mid h)$. The resulting next-token conditional distribution is exactly $p(\cdot\mid h)$. The same argument applies at the first position. By the chain rule, or induction on prefix length, $$P_{\rm alg}(X_{1:T}=x_{1:T})=\prod_{t=1}^T p(x_t\mid x_{1:t-1}).$$ A sequence with zero target probability is excluded by the corresponding zero conditional probability; arbitrary definitions at unreachable histories do not affect the joint law.

In a draft block, tokens after the first rejected position, and their verification probabilities, were computed using the original draft prefix. A correction changes that prefix. Later candidates generally no longer follow q conditioned on the actual prefix, and the old verification probabilities no longer give the required target conditionals. The standard algorithm must discard the remaining draft suffix and restart from the corrected output prefix. If all K candidates are accepted, the additional token is sampled from the target distribution conditioned on the full accepted prefix.

4(g). Why can speculative decoding be faster?

1. Under the stipulated independent, constant acceptance probability $\alpha=7/10$, $$P(L=\ell)=\alpha^\ell(1-\alpha),\quad0\le\ell<K;\qquad P(L=K)=\alpha^K.$$

ℓ	$P(L=\ell)$
0	$3/10$
1	$21/100$
2	$147/1000$
3	$343/1000$

These sum to $(300+210+147+343)/1000=1$.

2. For integer-valued $L\in\{0,\ldots,K\}$, $$L+1=\sum_{j=0}^K\mathbf1\{L\ge j\},\qquad P(L\ge j)=\alpha^j.$$ Taking expectations yields $$E[L+1]=\sum_{j=0}^K\alpha^j=\frac{1-\alpha^{K+1}}{1-\alpha}\quad(\alpha\ne1).$$ For $\alpha=1$, the expectation is $K+1$; for $\alpha=0$, it is 1. In this problem, $$E[L+1]=1+\frac7{10}+\frac{49}{100}+\frac{343}{1000}=\boxed{\frac{2533}{1000}=2.533}.$$ Equivalently, the probability table gives $1(0.3)+2(0.21)+3(0.147)+4(0.343)=2.533$.

3. The value 2.533 counts emitted tokens per verification round, rather than measuring wall-clock speedup. Two practical factors are:

1. Time spent generating K sequential draft candidates, plus residual computation, sampling, and scheduling overhead.
2. The cost of target verification, memory bandwidth, and KV-cache access or rollback. Verifying a block is not necessarily as cheap as decoding one token.

A rough cost model is $$\mathrm{speedup}\approx\frac{E[L+1]c_{\rm target}}{Kc_{\rm draft}+c_{\rm verify}(K)+c_{\rm overhead}}.$$ Acceleration requires sufficiently low extra costs. Real acceptance probabilities vary with the prefix and position and need not be independent, so the geometric sum applies to the simplified assumptions in this question.

Final synthesis (fewer than 100 words)

Uniform restriction underlies classical acceptance-rejection. Overlap acceptance reuses draft proposals with $P(Y=x,\mathrm{accept})=\min\{p(x),q(x)\}$. Residual correction supplies the missing mass because $p(x)=\min\{p(x),q(x)\}+(p(x)-q(x))_+$. This constructs a maximal coupling, with $P(\mathrm{accept})=1-D_{\mathrm{TV}}(p,q)$. Applying these identities at each actual prefix preserves the autoregressive target distribution. Cheap drafts enable parallel target verification; accepted prefixes amortize expensive target evaluations across multiple emitted tokens. Rejected suffixes are discarded and regenerated from the corrected prefix. Acceleration depends on acceptance rates and computational overhead, while the corrected output distribution remains exact.

Additional verification: exact fractions and speculative sampling

The answers and proofs in Question 4 do not depend on this section. Here, `Fraction` checks the identities exactly. Inverse-CDF sampling generates the draft and residual samples, and the plots compare final output frequencies with their target PMFs. Overlap acceptance with residual correction does not use a classical rejection envelope; the classical sampler's plot separately shows M^*q.

In [6]:
```
tokens = np.array(list('ABCD'))
p_exact = [F(2,5),F(3,10),F(1,5),F(1,10)]
q_exact = [F(1,5),F(2,5),F(1,10),F(3,10)]
m_exact = [min(a,b) for a,b in zip(p_exact,q_exact)]
d_exact = [max(a-b,F(0)) for a,b in zip(p_exact,q_exact)]
delta_exact=sum(d_exact)
tv_exact=sum(abs(a-b) for a,b in zip(p_exact,q_exact))/2
assert sum(m_exact)==F(7,10)
assert delta_exact==tv_exact==F(3,10)
```

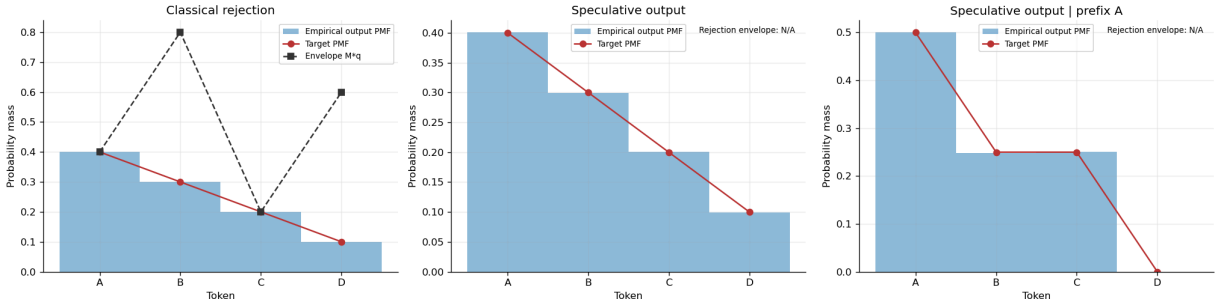
```
assert all(m+d==p for m,d,p in zip(m_exact,d_exact,p_exact))
assert sum(F(7,10)**j for j in range(4))==F(2533,1000)
table(['Token','p','q','Accepted mass','Residual r','Output'],
      [[t,str(p),str(q),str(m),str(d/delta_exact),str(m+d)]
       for t,p,q,m,d in zip(tokens,p_exact,q_exact,m_exact,d_exact)])

p0=np.array(p_exact,dtype=float); q0=np.array(q_exact,dtype=float)
def speculative_sample(n,p,q,generator=rng):
    p,q=np.asarray(p,dtype=float),np.asarray(q,dtype=float)
    assert np.all(p>=0) and np.all(q>=0)
    assert np.isclose(p.sum(),1) and np.isclose(q.sum(),1)
    y=inverse_finite(np.arange(len(q)),q,open_uniform(n,generator))
    acceptance=np.ones_like(p)
    np.divide(p,q,out=acceptance,where=q>0)
    acceptance=np.minimum(1,acceptance)
    accepted=generator.random(n)<=acceptance[y]
    x=y.copy()
    residual=np.maximum(p-q,0)
    if (~accepted).any():
        assert residual.sum()>0
        x[~accepted]=inverse_finite(np.arange(len(p)),residual/residual.sum(),
                                    open_uniform((~accepted).sum(),generator))
    return x,y,accepted

def classical_propose(n,generator):
    return inverse_finite(np.arange(4),q0,open_uniform(n,generator))
def classical_accept(y):
    return p0[y]/(2*q0[y])
x_classic, classic_proposals, classic_accepted=rejection_sample(N,classical_propose
x_spec,y_spec,spec_accept=speculative_sample(N,p0,q0)
pA=np.array([.5,.25,.25,0]); qA=np.array([.25,.5,0,.25])
x_next,y_next,next_accept=speculative_sample(N,pA,qA)
assert np.all(x_spec[spec_accept]==y_spec[spec_accept])
assert np.all(x_spec[~spec_accept]!=y_spec[~spec_accept])
fig,axes=plt.subplots(1,3,figsize=(17,4.4))
for ax,x,target,title in zip(axes,[x_classic,x_spec,x_next],[p0,p0,pA],
                             ['Classical rejection','Speculative output','Speculative
    ax.hist(x,bins=np.arange(-.5,4.5),weights=np.ones(len(x))/len(x),alpha=.5,label
    ax.plot(np.arange(4),target,'o-',color='#bd3434',label='Target PMF')
    if title=='Classical rejection':
        ax.plot(np.arange(4),2*q0,'s--',color='#333333',label='Envelope M*q')
    else:
        ax.text(.97,.95,'Rejection envelope: N/A',transform=ax.transAxes,ha='right'
    ax.set(xticks=np.arange(4),xticklabels=tokens,title=title,xlabel='Token',ylabel
    ax.legend(fontsize=8)
plt.tight_layout();plt.show()
table(['Quantity','Empirical','Theory'],[
    ['Classical acceptance',f'{classic_accepted/classic_proposals:.6f}','1/2'],
    ['Speculative acceptance',f'{spec_accept.mean():.6f}','7/10'],
    ['Conditional acceptance | A',f'{next_accept.mean():.6f}','1/2'],
    ['P(X2=C | X1=A)',f'{np.mean(x_next==2):.6f}','1/4'],
    ['P(X1=A, X2=C) via conditional estimate',f'{.4*np.mean(x_next==2):.6f}','1/10'
# Boundary case: identical distributions imply no residual and no rejection.
z, y, accepted = speculative_sample(1000,p0,p0,np.random.default_rng(123))
assert accepted.all() and np.array_equal(z,y)
# Disjoint support: every proposal rejects and residual restores the target.
```

```
z, y, accepted = speculative_sample(1000,[1,0],[0,1],np.random.default_rng(124))
assert not accepted.any() and np.all(z==0) and np.all(y==1)
print('Exact arithmetic, support, envelope and coupling checks passed.')
```

Token	p	q	Accepted mass	Residual r	Output
A	2/5	1/5	1/5	2/3	2/5
B	3/10	2/5	3/10	0	3/10
C	1/5	1/10	1/10	1/3	1/5
D	1/10	3/10	1/10	0	1/10



Quantity	Empirical	Theory
Classical acceptance	0.499350	1/2
Speculative acceptance	0.700090	7/10
Conditional acceptance | A	0.497530	1/2
P(X2=C | X1=A)	0.251180	1/4
P(X1=A, X2=C) via conditional estimate	0.100472	1/10

```
Exact arithmetic, support, envelope and coupling checks passed.
```

Experimental interpretation. Classical acceptance is approximately $1/2$ and speculative acceptance is approximately $7/10$, while both output histograms agree with the same target PMF. With prefix A, the draft never proposes C, but approximately $1/4$ of corrected outputs are C. This demonstrates the role of residual correction. The final identical-distribution and disjoint-support checks cover the extreme cases of no rejections and universal rejection.

All empirical quantities have Monte Carlo error. Frequency errors typically scale as $O(n^{-1/2})$, so empirical TV distances need not vanish. The analytical arguments establish correctness, global envelope validity, and preservation of the target distribution. The assignment source is `hw2.pdf` in the same directory. All derivations and implementations are provided within this notebook; no external datasets or model downloads are required.